LOOP STATEMENT COMPILER

PROJECT REPORT

24CST303 COMPILER DESIGN

Submitted

By

1. Daniya Dennis,AJC24CS079
2. Irene Tesssa Seban,AJC24CS108
3. Jithu Joesph,AJC24CS115
4. Karthik Raj,AJC24CS128
5.Kevin Kanniyakonil Biju,AJC24CS130
6.Maria Rose Johny,AJC24CS140

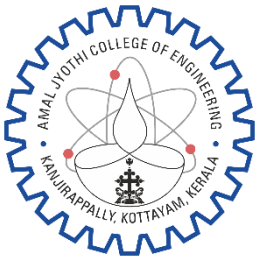
In partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

AMAL JYOTHI COLLEGE OF ENGINEERING (AUTONOMOUS), KANJIRAPALLY 686 518

October 2026

AMAL JYOTHI COLLEGE OF ENGINEERING
(AUTONOMOUS), KANJIRAPALLY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONA FIDE CERTIFICATE

This is to certify that the Project Report entitled "LOOP STATEMENT COMPILER" submitted by 1.Daniya Dennis, 2.Irene Tessa Seban, 3.Jithu Joseph, 4.Karthik Raj,5.Kevin Kanniyakonil Biju,6.Maria Rose Johny in partial fulfillment of the requirement for the award of the Bachelor of Technology in Computer Science and Engineering is a bona fide record of the work carried out at AMAL JYOTHI COLLEGE OF ENGINEERING (AUTONOMOUS), under our guidance and supervision.

INTERNAL EXAMINER
NAME :Dr.Sinciya P.O
DESIGNATION :Assistant Professor
DEPT :Computer Science Engineering

HEAD OF THE DEPARTMENT
NAME:Dr.Juby Mathew
DESIGNATION :HOD
DEPT :Computer Science Engineering

ACKNOWLEDGMENT

We would like to express our heartfelt gratitude to our project guide Dr.Sinciya P.O, for the invaluable support, guidance, and encouragement provided throughout the development of our project, Loop Statement Compiler. Their insights and suggestions have been instrumental in the successful completion of this project.

We are grateful to the management and faculty of **Amal Jyothi College of Engineering (Autonomous), Kanjirappally**, especially the Department of Computer Science and Engineering, for providing us with the facilities, resources, and academic support necessary to carry out this project.

We would also like to thank our Head of the Department, Dr.Juby Mathew, and all the faculty members who provided guidance and encouragement during the course of this work.

Finally, we extend our sincere thanks to our teammates, classmates, friends, and families for their cooperation, motivation, and continuous support throughout the project.

ABSTRACT

The Loop Statement Compiler is a mini compiler front-end developed to analyze and validate loop statements in the C programming language. The project focuses on three fundamental looping constructs: `for`, `while`, and `do-while`. The primary objective is to demonstrate the basic stages of compiler front-end development, including lexical analysis, syntax validation, semantic checking, error detection, and tree-based representation of source code.

The system consists of a C-based compiler component integrated with a web-based user interface developed using HTML, CSS, JavaScript, and Flask. Users can enter C source code through the application interface and initiate compilation to examine the results. The system identifies lexical tokens such as keywords, identifiers, operators, constants, and delimiters, and displays them along with their source positions. It validates the structure of loop statements. Basic semantic checks are also performed to identify selected errors.

The application provides a tree-style representation of recognized program structures and displays diagnostic messages when errors are detected. Its interface organizes the compilation results into separate sections for tokens, AST-style output, errors and warnings, and compilation logs. Testing with valid and invalid inputs demonstrates the system's ability to process supported loop constructs and identify selected syntax and semantic errors.

The project provides practical insight into compiler-front-end concepts and illustrates how source-code analysis can be integrated with an interactive application interface.

Keywords: Mini Compiler, Compiler Front-End, Lexical Analysis, Syntax Analysis, Semantic Analysis, Loop Statements, Parse Tree, Abstract Syntax Tree, Error Handling.

TABLE OF CONTENTS

Chapter No:	Title	Page no:
	Acknowledgement	**3**
	Abstract	**4**
1.	**Introduction** 1.1 Introduction 1.2 Problem Statement 1.3 Objectives 1.4 Scope of the Project	**6**
2.	**System Architecture and Design** **2.**1 System Architecture 2.2 Context Free Grammar 2.3 Grammar Transformations	**8**
3.	**Implementation and Testing** 3.1 Implementation Overview 3.2 Lexical Analysis 3.3 Syntax Analysis and Loop Validation 3.4 Semantic Checks and Error Handling 3.5 Parse Tree Output 3.6 Test Cases and Results 3.7 Screenshots	**11**
4.	**Conclusion** 4.1 Conclusion 4.2 Limitations	**23**

CHAPTER 1

INTRODUCTION

1.1. INTRODUCTION

A compiler translates a program written in a high-level language into a form that a computer can execute. The front end of a compiler examines the source program and checks whether its words, symbols and statement structures follow the language rules. Lexical analysis groups characters into tokens, syntax analysis checks the arrangement of those tokens, and semantic analysis checks selected meaning-related rules such as whether a variable has been declared.

Loop statements are important in C because they allow a group of statements to execute repeatedly. A for loop normally contains initialization, a condition and an update expression. A while loop checks its condition before executing the body, while a do-while loop executes the body before checking the condition. Because their headers and control flow differ, these statements provide a useful topic for demonstrating compiler front-end concepts.

The supplied project contains a C compiler component and a browser-based interface. The C component prints tokens, validates selected syntax and semantic properties, and produces a parse-tree-style text output. The web application uses HTML, CSS and JavaScript for the interface and a Python Flask backend with an adapter to invoke the native compiler or use a clearly labelled fallback mode when the executable is unavailable.

1.2. PROBLEM STATEMENT

Incorrect loop syntax can cause compilation errors and make a program difficult to understand or execute. Examples include missing semicolons in a for-loop header, missing closing parentheses, unmatched braces, malformed loop conditions, missing update expressions and the use of variables that have not been declared. The aim of this project is to develop a small compiler front end that analyses C code containing for, while and do-while loops, detects selected errors, and presents diagnostic information and a readable tree-like output.

1.3. OBJECTIVES

- To identify tokens in the input source code, including keywords, identifiers, literals, operators and delimiters.

- To validate the structure of for, while and do-while loop statements.
- To check selected loop-header components, including initialization, conditions and increment/decrement expressions.
- To perform basic semantic checks, especially checking whether identifiers have been declared.
- To detect selected structural errors such as missing punctuation and unmatched braces.
- To display a parse-tree-style representation of the program when the implemented checks pass.
- To provide a web interface for entering source code and viewing tokens, tree output, errors and logs.
- To demonstrate the main stages of a compiler front end in a small educational project.

1.4. Scope of the Project

The project focuses on a limited subset of C syntax needed to demonstrate loop analysis. It is not a complete C compiler and does not generate machine code. The checks are based on the rules implemented in the supplied C source, so the report should not claim complete support for every valid C expression or all C language rules. The tree output is generated by scanning the source and printing a hierarchical text representation; it should be described as parse-tree-style output unless the implementation is extended to construct a full AST data structure.

CHAPTER 2

SYSTEM ARCHITECTURE AND DESIGN

2.1. System Architecture

The supplied application follows this high-level data flow:

1. User	Enters or selects a C loop program in the browser interface.
2. Web Front End	HTML, CSS and JavaScript provide the editor, Compile control and output panels.
3. Flask Backend	Receives the source code through the compile endpoint and validates the request.
4. Compiler Adapter	Finds and invokes the native C compiler, or uses the documented fallback mode if needed.
5. C Compiler Component	Performs token scanning, selected syntax/semantic checks, diagnostics and tree-style output.
6. Results	Tokens, tree output, diagnostics and logs are returned to the interface.

Figure 1. System architecture (textual representation). For the final report, this table may be converted into a block diagram with arrows between the components.

2.2. Context-Free Grammar (CFG)

A Context-Free Grammar describes the legal structure of statements using terminals, non-terminals and production rules. The following CFG is a proposed simplified grammar for the loop constructs in this project. It describes the intended subset of C syntax and is a documentation model; it is not a claim that the supplied implementation uses a formal parser generator.

2.2.1 Grammar Symbols

- Terminals: for, while, do, (,), {, }, ;, =, ++, --, relational and arithmetic operators.
- Non-terminals: <program>, <statement>, <loop>, <for-loop>, <while-loop>, <do-while-loop>, <body>, <init>, <condition>, <update>, <expression>.
- ε denotes an empty production, where allowed.

2.2.2 Production Rules

```
<program>          ::= <statement-list>
<statement-list>   ::= <statement> <statement-list> | ε
<statement>        ::= <loop> | <simple-statement> | <block>
<loop>             ::= <for-loop> | <while-loop> | <do-while-loop>

<for-loop>         ::= for ( <init> ; <condition> ; <update> ) <body>
<while-loop>       ::= while ( <condition> ) <body>
<do-while-loop>    ::= do <body> while ( <condition> ) ;

<body>             ::= <statement> | { <statement-list> }
<init>             ::= <expression> | <declaration>
<update>           ::= <expression>
<condition>        ::= <expression>
<block>            ::= { <statement-list> }
<simple-statement>::= <expression> ; | <declaration> ;

<declaration>      ::= <type> identifier [ = <expression> ]
<type>             ::= int | float | char | double
<expression>       ::= identifier | number | <expression> <operator>
<expression>
<operator>         ::= + | - | * | / | < | > | <= | >= | == | != | =
| ++ | --
```

2.2.3 CFG for the Three Loop Statements

For loop:

```
<for-loop>
    ├── for
    ├── (
    ├── <init>
    ├── ;
    ├── <condition>
    ├── ;
    ├── <update>
    ├── )
    └── <body>
```

While loop:

```
<while-loop>
    ├── while
    ├── (
    ├── <condition>
    ├── )
    └── <body>
```

Do-while loop:

```
<do-while-loop>
    ├── do
    ├── <body>
    ├── while
    ├── (
    ├── <condition>
    ├── )
    └── ;
```

2.3. Grammar Transformations

Grammar transformations are applied when needed to make a grammar suitable for a parsing strategy. Two common transformations are left factoring and elimination of left recursion. The supplied C code scans the source and performs checks through hand-written functions; it does not define a formal grammar or show a parser-generator grammar file. Therefore, the transformations below are relevant grammar-design techniques for the documented CFG, not transformations already performed by the program.

2.3.1 Left Factoring

Left factoring is useful when two or more productions begin with the same prefix. The common prefix is extracted so that the parser can decide which alternative to use after reading more input. In the simplified loop grammar, the three loop productions begin with different keywords (for, while and do), so left factoring is not required for selecting among these three loop types.

2.3.2 Elimination of Left Recursion

Left recursion occurs when a non-terminal can derive a string beginning with itself, such as <expression> ::= <expression> + <term>. A recursive-descent parser can have difficulty with direct left recursion. One common transformation is to rewrite the grammar using a base production followed by a repeatable tail.

Before:

```
<expression> ::= <expression> + <term> | <term>
```

After:

```
<expression> ::= <term> <expression-tail>
<expression-tail> ::= + <term> <expression-tail> | ε
```

This is an illustrative expression-grammar transformation. It should only be included as a required project transformation if the instructor expects a transformation example; it is not evidence that the supplied C program implements this exact grammar.

CHAPTER 3

IMPLEMENTATION AND TESTING

3.1 IMPLEMENTATION OVERVIEW

This section describes the implementation of the Loop Statement Compiler, a mini compiler front end designed to analyze a limited subset of C programs containing for, while and do-while loops. The project combines a C compiler component with a browser-based interface. The C component performs source scanning, selected syntax and semantic checks, diagnostics, and parse-tree-style text output. The web application provides a code editor and organized views for tokens, tree output, errors and warnings, and raw logs.

3.1.1 WEB INTERFACE AND COMPILER INTEGRATION

The browser interface is built with HTML, CSS and JavaScript. The interface shows a source editor, an example-selection menu, Clear and Copy controls, and a results area containing Tokens, AST, Errors & Warnings, and Raw Logs views. The token view presents token number, token type, lexeme, line number, and column number. A status indicator summarizes the reported compilation result.

The example menu includes valid for-loop, while-loop, and do-while-loop examples, nested loops, and selected syntax-error examples. These examples support repeatable demonstrations. The presence of a preset demonstrates that the interface supplies the example; successful validation must be established by compiling it and checking the returned results.

The application architecture uses a Flask backend to receive source code and a compiler adapter to invoke the native C executable, with a documented fallback mode if the executable is unavailable. For a test result to be attributed to the C compiler, the run should be confirmed as using the native compiler rather than fallback mode.

3.2 LEXICAL ANALYSIS

Lexical analysis identifies meaningful units in source code, such as keywords, identifiers, literals, operators, and delimiters. In this project, lexical information is presented in the Tokens view. The displayed token records include a token type and lexeme together with the source line and column, allowing a user to locate the corresponding element in the editor.

The supplied interface screenshot shows examples such as KEYWORD for int, while, printf and return; IDENTIFIER for i; OPERATOR for = and ++; NUMBER for

numeric constants; STRING for a string literal; and DELIMITER for punctuation such as parentheses, braces, commas and semicolons. A preprocessor line such as #include <stdio.h> is also shown as a PREPROCESSOR token.

3.3 Syntax Analysis and Loop Validation

Syntax analysis checks whether source elements appear in an arrangement accepted by the implemented rules. The compiler focuses on the headers and bodies of for, while, and do-while loops, together with selected delimiters and statement structure.

3.3.1 for Loop

A for loop has an initialization expression, a condition, and an update expression in its header, separated by semicolons. The body follows the closing parenthesis.

```
for (initialization; condition; update) {\n    statements;\n}
```

The compiler checks selected properties of the header and body, including the update expression and structural delimiters. A representative valid input is:

```
int k;\nfor (k = 0; k < 10; k++) {\n    k = k * 2;\n}
```

The supplied console output for this example reports that the code is syntactically and semantically valid and prints a tree-style representation containing the declaration, initialization, condition, update, and body statement.

3.3.2 while Loop

A while loop evaluates its condition before each iteration. Its general form is:

```
while (condition) {\n    statements;\n}
```

A representative input is:

```
int j;\nwhile (j > 5) {\n    j = j - 1;\n}
```

The supplied console output accepts this example and prints a tree-style representation showing the condition and body statement. The error example with a missing closing brace demonstrates that the compiler can report selected structural problems. The compiler's checks remain limited to the implemented subset of C.

3.3.3 DO-WHILE LOOP

A do-while loop executes its body before evaluating the condition. Unlike for and while loops, the complete construct ends with a semicolon after the condition.

```
do {\n    statements;\n} while (condition);
```

The do-while validates the body, the trailing while keyword, the parenthesized condition, and the terminating semicolon, and should associate the body and condition with the same loop in its output. Include a successful compilation screenshot from the corrected version as evidence..

3.4 SEMANTIC CHECKS AND ERROR HANDLING

Semantic analysis checks selected meaning-related properties beyond the arrangement of symbols. The implementation includes basic variable-declaration checking and selected checks for update expressions. These checks are useful for demonstrating semantic analysis, but they do not amount to full C type checking or scope analysis.

3.4.1 IDENTIFIER DECLARATION CHECK

The implementation maintains a simple symbol table for recognized declared variables. When an identifier is encountered in an expression, the declaration-checking logic checks whether that name has been recorded. If an identifier is not found, the compiler can report an undeclared-variable error. This is a basic semantic check; the report should not imply complete handling of C scopes, shadowing, function parameters, or all declaration forms unless those are implemented.

3.4.2 UPDATE-EXPRESSION CHECK

The implementation performs selected checks on for-loop update expressions. It detects an empty update expression, certain invalid operator combinations, and whether the expression contains one of the update-related characters recognized by the current validation logic. These checks are intentionally limited and should not be described as complete validation of all C expressions.

3.4.3 STRUCTURAL ERRORS AND DIAGNOSTICS

The compiler reports selected structural problems such as unmatched braces and missing elements in loop headers. These are primarily syntax or structural errors, while undeclared identifiers are semantic errors. The frontend organizes diagnostics under Errors & Warnings and Raw Logs, while the status indicator summarizes the reported outcome.

A success message means that the input passed the checks performed by the current implementation. It does not prove that the input satisfies every rule of the C language. Likewise, a test should be described as detecting a particular error only when that behavior is reproduced in the final submitted version.

3.5 Parse-Tree Output

The application includes a tab labelled AST. In the report, the output is described as a parse-tree-style representation unless the final source constructs and traverses a formal abstract syntax tree data structure. The earlier C console output prints a PROGRAM root and indented entries for declarations, loop constructs, loop-header components, and body statements.

For a for loop, the displayed components include initialization, condition, update, and body. For a while loop, the output includes the condition and body. The corrected do-while implementation should show the body and trailing condition as components of the same do-while construct. Capture the actual final output for each loop type rather than assuming the display is correct from the tab name alone.

A textual tree helps demonstrate the structural organization of recognized source code. A formal AST would normally use explicit node objects and parent-child links, omitting some grammar-only punctuation. Whether the current implementation qualifies as a formal AST depends on its actual data structures, not on the interface label.

3.6 Test Cases and Results

The following test plan combines the examples visible in the interface with the console tests supplied during development.

Test ID	Input / Scenario	Expected result	Evidence
TC-01	Valid for loop with declaration, condition, update and body	Accepted; tree-style output shows initialization, condition, update and body.	Figure 3.7.6
TC-02	Valid while loop with condition and body	Accepted; tree-style output shows condition and body.	Figure 3.7.7
TC-03	Valid do-while loop ending with while (condition);	Accepted; body and trailing condition are validated and represented together.	Figure 3.7.8

TC-04	Invalid for missing operand	Rejected with an missing operand diagnostic.	Figure 3.7.12
TC-05	Missing closing brace in a while loop	Rejected with a structural error.	Figure 3.7.11
TC-06	Missing semicolon after do-while condition	Rejected with a syntax diagnostic.	Figure 3.7.10
TC-07	Nested for and while loops	Accepted if supported; tree representation should preserve nesting.	Figure 3.7.9

Table 4.1. Test cases for the Loop Statement Compiler.The screenshots supplied in section 3.7 confirm the outcomes of TC-01, TC-02, TC-03 and TC-07.

3.7. SCREENSHOTS

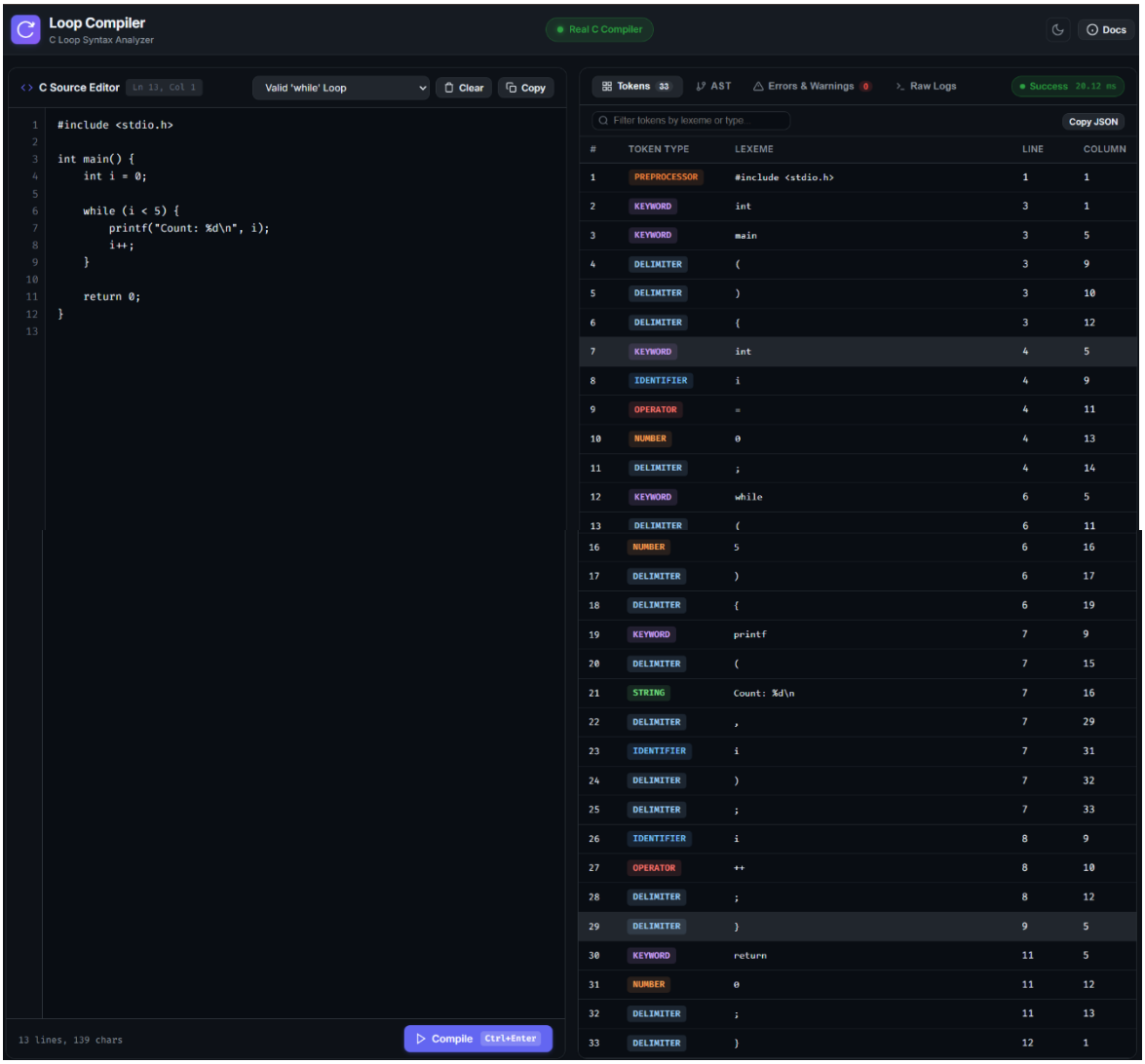


Figure 3.7.1. Web interface showing the C source editor and token table.

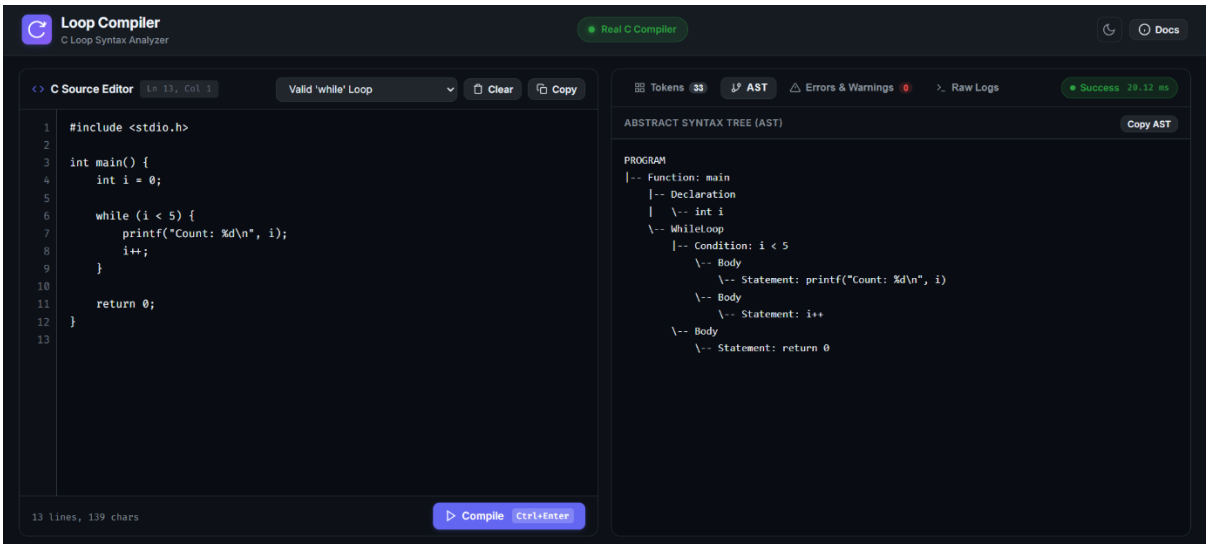


Figure 3.7.2. AST tab showing the tree-style output for a valid for or while loop.

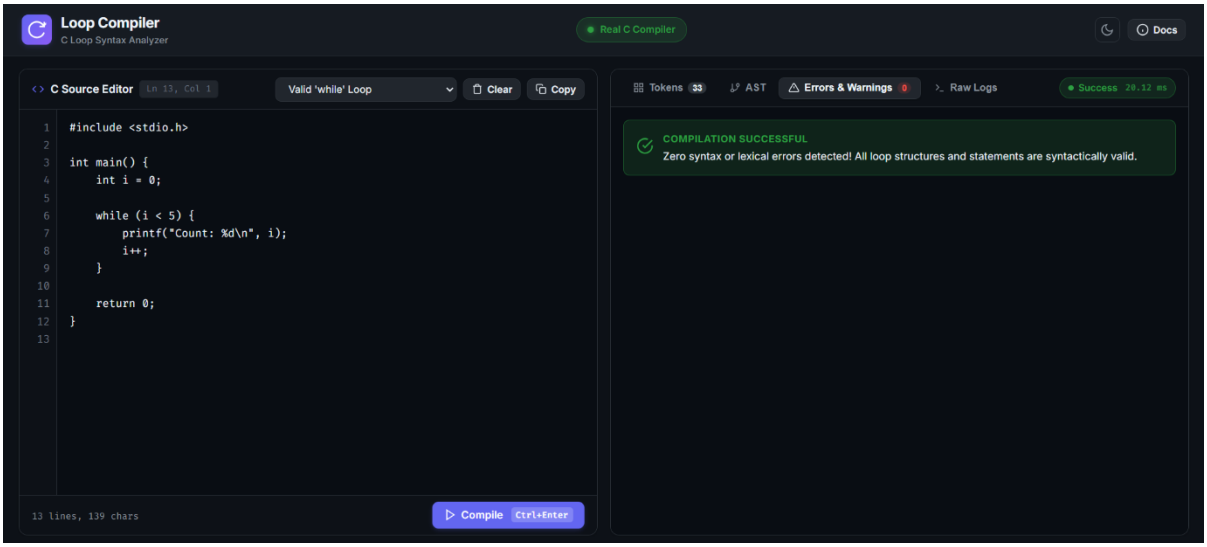


Figure 3.7.3 Successful while compilation message in compilation-status indicator.

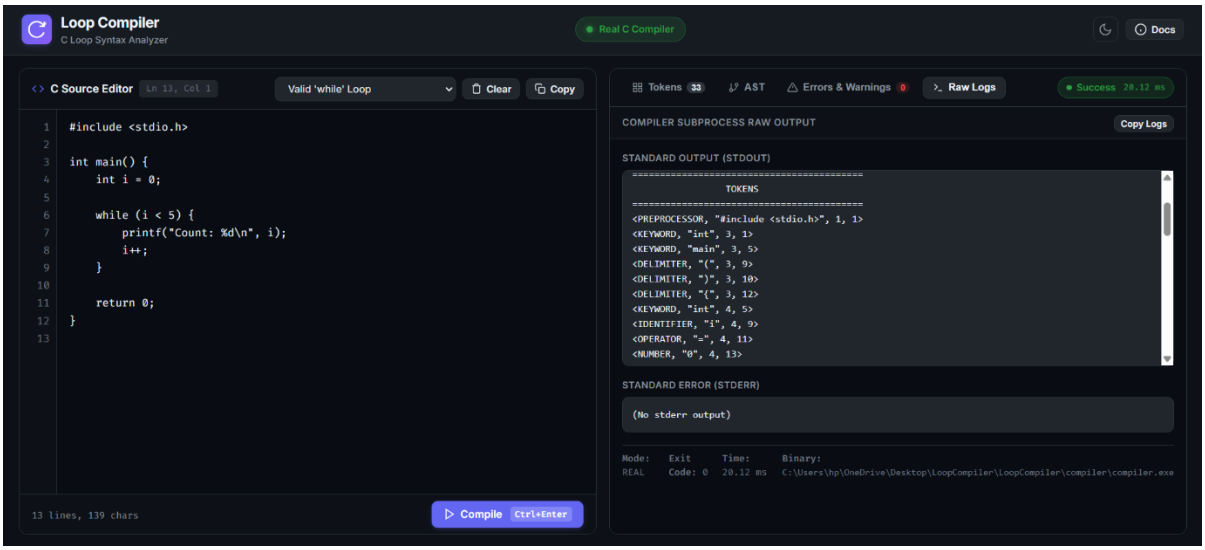


Figure 3.7.4 Tokens in raw compilation logs

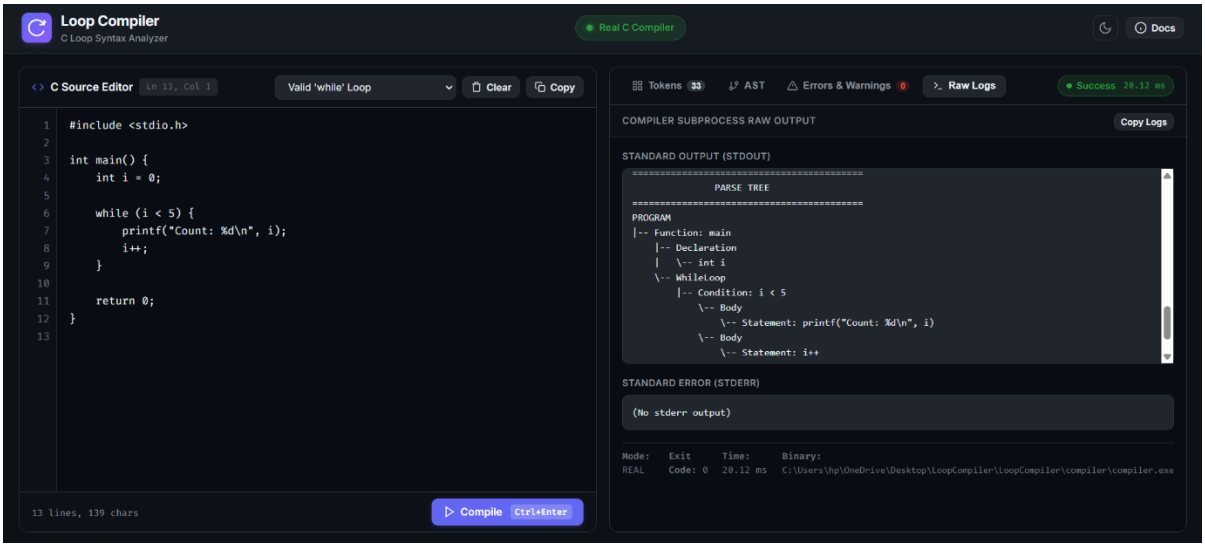


Figure 3.7.5 Parse Tree in raw compilation logs

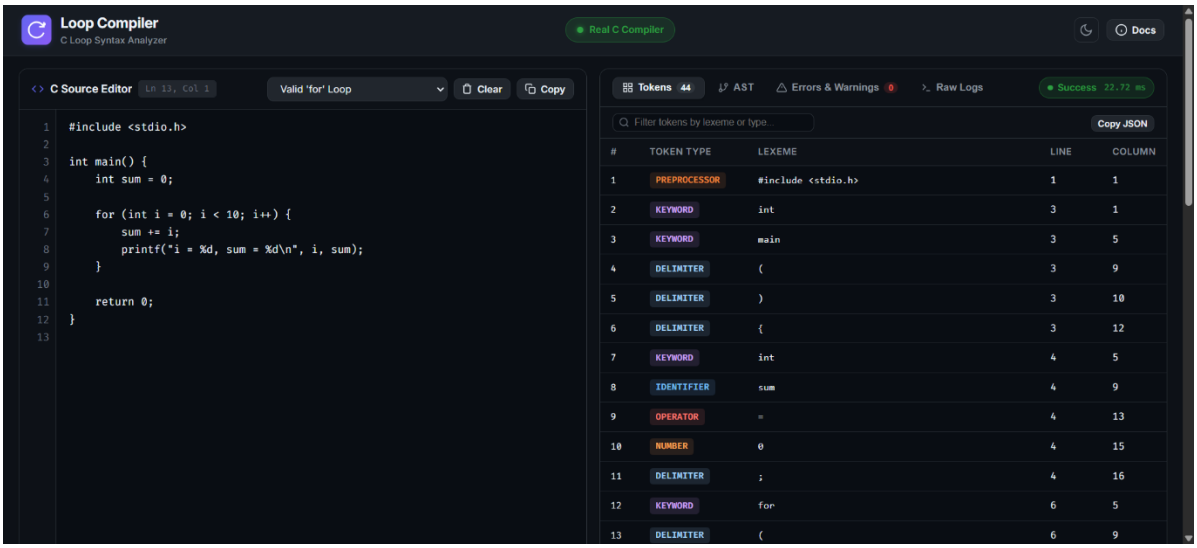


Figure 3.7.6 Valid for loop

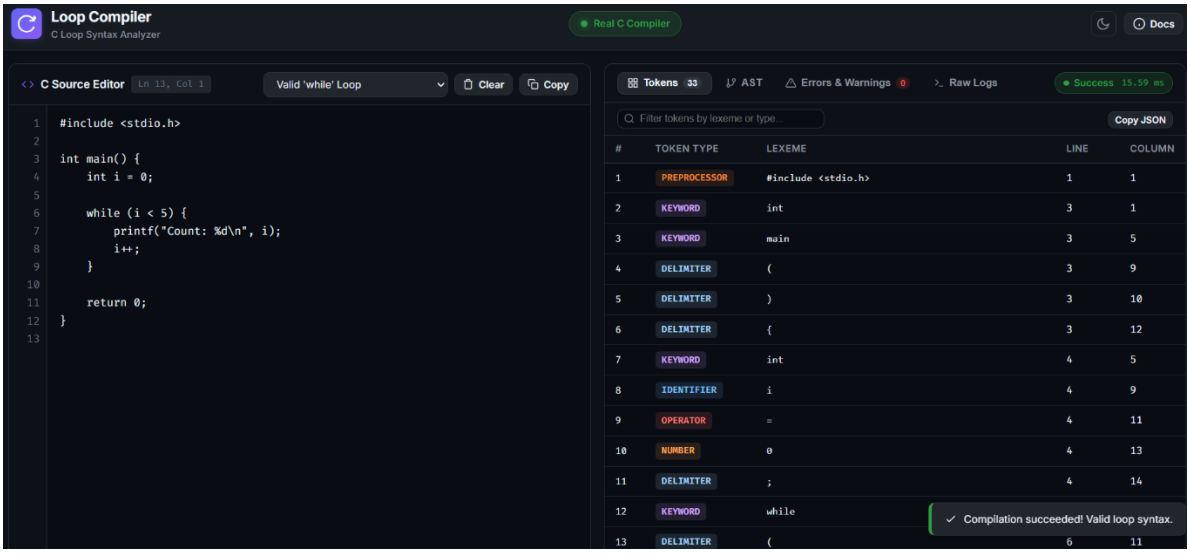


Figure 3.7.7 Valid while loop

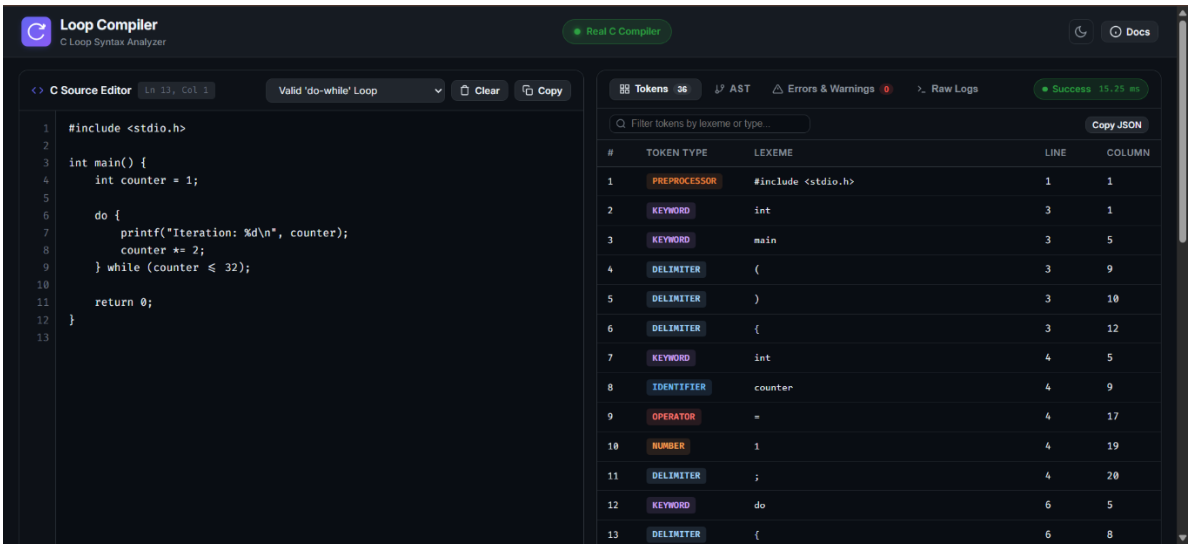


Figure 3.7.8 Valid do-while loop

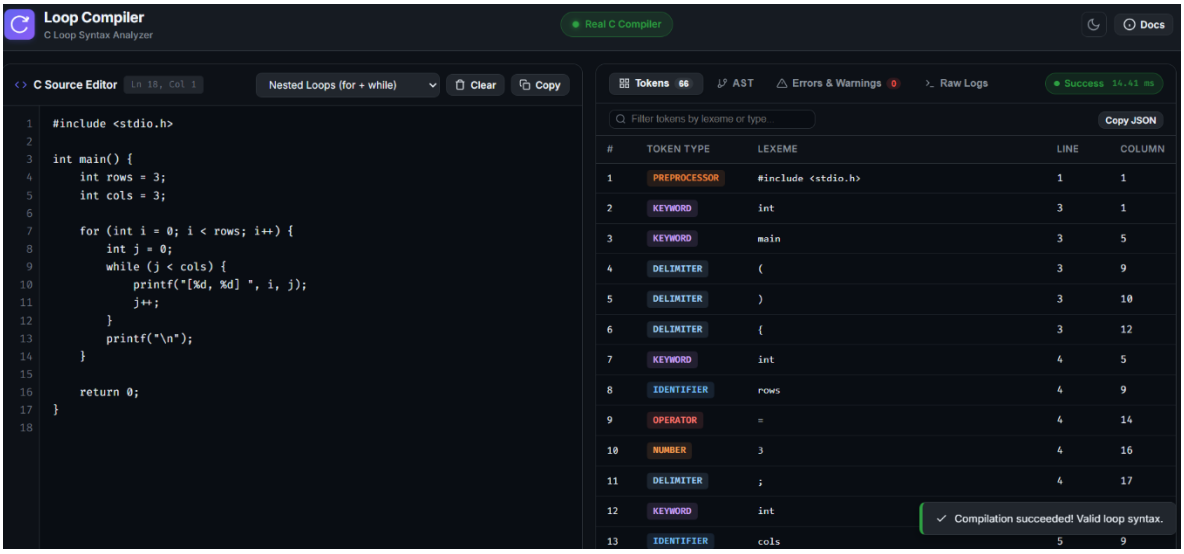


Figure 3.7.9 Valid nested loop

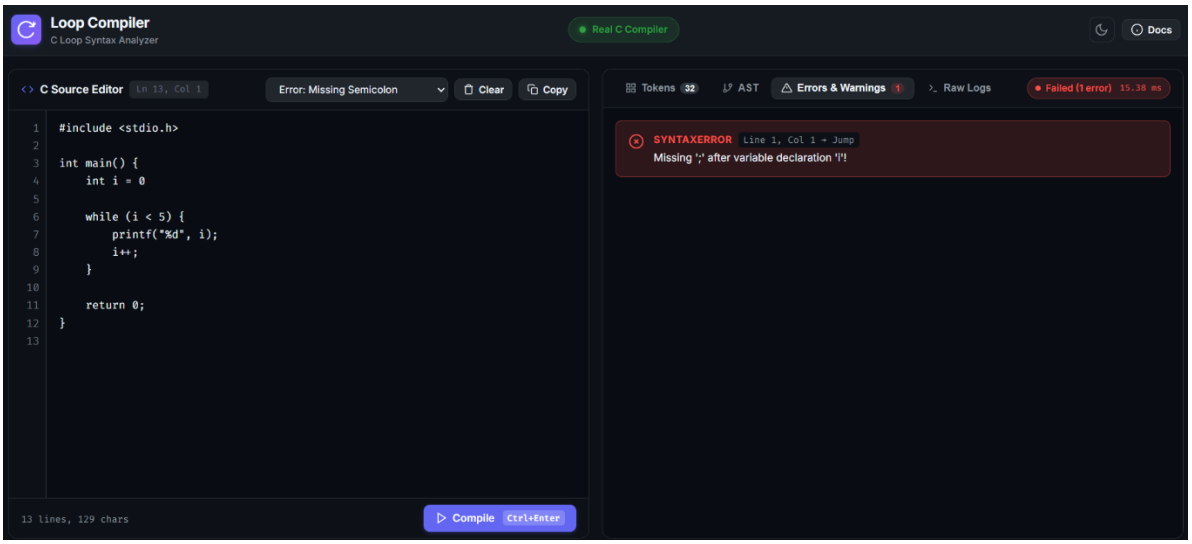


Figure 3.7.10 Error and warning output for an invalid loop, such as a missing semicolon

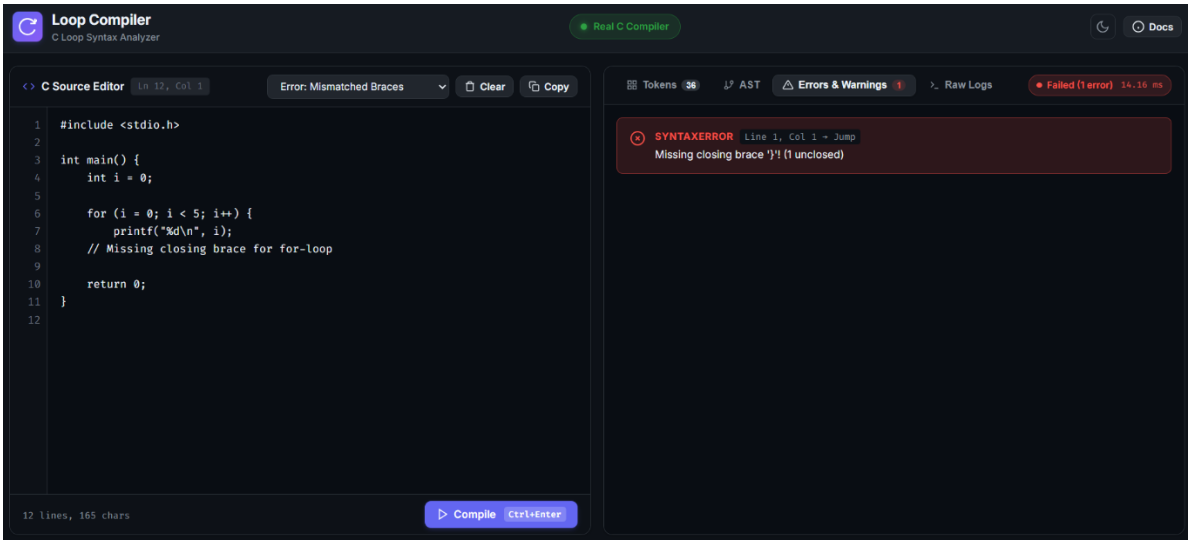


Figure 3.7.11 Error and warning output for an invalid loop, such as a missing brace

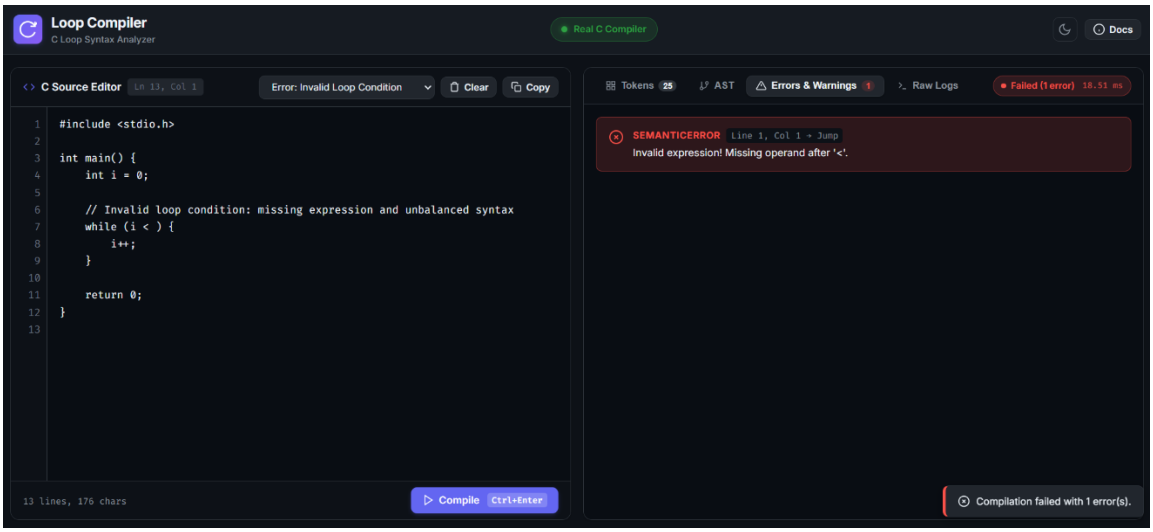


Figure 3.7.12 Error and warning output for an invalid loop, such as a missing operand

3.8 Results and Discussion

The project combines a browser-based interface with a C-based compiler component to demonstrate selected compiler-front-end tasks. The interface provides a source editor and organized result panels, while the compiler component performs the checks implemented in the submitted source and returns diagnostic and tree-style output.

The supplied console tests demonstrate successful handling of representative for and while loops, along with detection of an invalid update expression and a missing closing brace. The team reports that the do-while validation issue has since been corrected. The final report should include a successful do-while run from the corrected version and tests for the required trailing semicolon and condition before describing this functionality as verified.

CHAPTER 4
CONCLUSION

4.1 CONCLUSION

The Loop Statement Compiler demonstrates key compiler-front-end concepts through a C-based validation component and a browser interface. It presents lexical information, performs selected syntax and semantic checks, reports errors, and displays a parse-tree-style representation for supported loop constructs.

The supplied tests demonstrate successful processing of representative for and while loops and detection of selected errors. The team has also corrected the do-while validation issue; its successful behavior should be documented using tests run against the final submitted source. The project provides a practical educational demonstration of compiler analysis while retaining the limitations of a prototype that supports only a subset of C.

.

4.2 LIMITATIONS

- The project supports a limited subset of C rather than the complete C language.
- The CFG describes the intended language subset and is not itself executed by a formal parser.
- Semantic analysis is limited; comprehensive type checking, scope rules, assignment compatibility and all declaration forms are not implemented.
- The AST tab label should not be taken as proof of a formal AST data structure; describe the output according to the actual implementation.
- Nested-loop behavior and complex expressions require testing because simple scanning and shared parser state may not handle every valid case.
- Fixed-size buffers and limited input checks may affect robustness for large or unusual inputs.
- The web interface depends on correct backend integration. Results from fallback mode, if used, must be distinguished from results generated by the native C compiler.